BP101T COMPANION TEXT

Python for Pharmacists

Unit III · Data Structures & File Handling

BioDockify

Building the Future of AI-Driven Pharmaceutical Research

learn.biodockify.com

Course BP101T · B.Pharm Semester I

Chapters 9–12 · Six teaching hours

Theory-first · 49 code examples, outputs machine-verified · NumPy, CSV files, healthcare datasets

Basics of Python Programming for Pharmaceutical Sciences

Contents

Chapter 9 — Lists and Tuples....1
9.1 Why collections exist....1
9.2 Creating, indexing and slicing lists....1
9.3 List methods — adding, removing, sorting....2
9.4 List comprehensions — one-line list building....4
9.5 Nested lists — a table in a variable....5
9.6 Tuples — the sealed record....5
Chapter 10 — Dictionaries and String Tools....7
10.1 Dictionaries — data with names on it....7
10.2 Adding, updating and removing....7
10.3 Looping through a dictionary....9
10.4 Nested dictionaries — a small patient database....9
10.5 Dictionary comprehension....9
10.6 String manipulation techniques (syllabus point)....10
Chapter 11 — NumPy Arrays....11
11.1 What NumPy is and why it exists....11
11.2 Creating arrays....11
11.3 Element-wise arithmetic — the vectorized advantage....13
11.4 Aggregation functions....13
11.5 Slicing and boolean filtering....13
Chapter 12 — CSV Files and Healthcare Datasets....15
12.1 What CSV is....15
12.2 Reading a CSV file — the csv module....16
12.3 Writing a CSV file....16
12.4 DictReader — rows as dictionaries....17
12.5 Handling a missing file safely....17
12.6 Structured healthcare datasets (syllabus point)....18
12.7 Importing a pharmaceutical dataset — the complete workflow....18

Unit III — Consolidation.....19

Syllabus coverage map.....19

Twenty-question exercise bank.....19

Answers to selected exercises.....22

Important points for examination.....23

Right-click the table of contents and choose “Update Field” to refresh page numbers.

Unit hours: 6 · **Book chapters:** 9–12 · **Syllabus bullets covered:** lists, tuples, and dictionaries; indexing and slicing lists, basic operations on lists and dictionaries, string manipulation techniques; introduction to NumPy arrays, basic operations using NumPy (array creation, arithmetic operations); reading and writing CSV files; understanding structured healthcare datasets; importing small pharmaceutical datasets and performing basic data access and manipulation tasks.

Unit objectives. After studying this unit you should be able to: (1) store collections of values in lists, tuples and dictionaries; (2) index, slice and manipulate those collections; (3) apply string-manipulation techniques to pharmacy text; (4) create and compute with NumPy arrays; (5) read and write CSV files; (6) recognise structured healthcare datasets and run basic access and manipulation on them.

Chapter 9 — Lists and Tuples

9.1 Why collections exist

Units I–II gave every value its own variable — fine for one patient's dose, hopeless for a ward of forty. A **collection** stores many values inside one named variable, and the three you will meet — list, tuple, dictionary — differ in exactly two ways: can you *change* them after creation (*mutability*), and how do you *address* an item (by position or by key)? Choose the right one and the program almost writes itself.

9.2 Creating, indexing and slicing lists

Think of a **list** as a numbered, editable collection inside square brackets. Items keep their insertion order, may repeat, and may mix types.

```
drugs = ["Paracetamol", "Ibuprofen", "Aspirin", "Cetirizine"]
prices = [12.5, 8.0, 5.5, 15.0]
mixed = ["Tab", 500, True]        # any type may live in a list
empty = []                         # starts empty, grows later
print(len(drugs))
```

OUTPUT

```
4
```

Indexing and slicing work exactly as they do on strings (Unit I, §4.2–4.3): positions count from zero, negative positions count from the end, slices exclude the stop, and over-generous slices stop gracefully.

```
print(drugs[0], "|", drugs[-1])
print(drugs[1:4])
print(drugs[:3])
```

```
print(drugs[2:])
print(drugs[::-1])
```

OUTPUT

```
Paracetamol | Cetirizine
['Ibuprofen', 'Aspirin', 'Cetirizine']
['Paracetamol', 'Ibuprofen', 'Aspirin']
['Aspirin', 'Cetirizine']
['Cetirizine', 'Aspirin', 'Ibuprofen', 'Paracetamol']
```

Five slicing patterns to memorise: `[a:b]` a range, `[:b]` from the start, `[a:]` to the end, `[::-1]` reversed, `[::2]` every second item.

```
stock = [50, 30, 80, 15, 60]
print(stock[::2])          # every 2nd
print(stock[:-2])          # all except the last 2
print(stock[1:4:2])        # indices 1 and 3
```

OUTPUT

```
[50, 80, 60]
[50, 30, 80]
[30, 15]
```

Reading past the end raises `IndexError` — the last valid position is `len(list) - 1`; a slice past the end simply returns what exists.

9.3 List methods — adding, removing, sorting

Method	What it does	Example
`.append(item)`	adds to the end	`drugs.append("Metformin")`
`.insert(i, item)`	inserts at position i	`drugs.insert(1, "Amoxicillin")`
`.remove(item)`	removes the first match	`drugs.remove("Aspirin")`
`.pop(i)`	removes and returns position i (last by default)	`drugs.pop(0)`
`.sort()`	sorts ascending, in place	`prices.sort()`
`.reverse()`	reverses, in place	`drugs.reverse()`
`.count(item)`	counts occurrences	`stock.count(50)`
`.index(item)`	position of the first match	`drugs.index("Aspirin")`
`.extend(list2)`	appends all of list2	`drugs.extend(more)`
`.clear()`	empties the list	`stock.clear()`

Five examples:

```
drugs = ["Paracetamol", "Ibuprofen", "Aspirin"]
```

```
drugs.append("Cetirizine")
print(drugs)
```

OUTPUT

```
['Paracetamol', 'Ibuprofen', 'Aspirin', 'Cetirizine']
```

```
stock = [50, 30, 80, 15]
stock.sort()
print(stock)
print(sum(stock), max(stock), min(stock))
```

OUTPUT

```
[15, 30, 50, 80]
175 80 15
```

```
drugs = ["Paracetamol", "Ibuprofen", "Aspirin", "Ibuprofen"]
print(drugs.count("Ibuprofen"))
print(drugs.index("Aspirin"))
```

OUTPUT

```
2
2
```

```
list1 = ["Aspirin", "Dolo"]
list2 = ["Brufen", "Meftal"]
list1.extend(list2)
print(list1)
```

OUTPUT

```
['Aspirin', 'Dolo', 'Brufen', 'Meftal']
```

```
stock = [50, 30, 80]
print("Aspirin" in drugs)
removed = stock.pop()
print(removed, stock)
```

OUTPUT

```
True
80 [50, 30]
```

The copying pitfall. `new = old` does **not** copy a list — both names refer to one shared object, so editing through either name alters the single list. Use `new = old.copy()` (or `list(old)`) for a genuine independent copy.

9.4 List comprehensions — one-line list building

A **list comprehension** constructs a fresh list in a single expression, standing in for a three-line loop:

```
doses = [250, 500, 750, 1000]
```

```
doubled = [d * 2 for d in doses]
print(doubled)
```

OUTPUT

```
[500, 1000, 1500, 2000]
```

Add a condition to filter while building:

```
high = [d for d in doses if d >= 500]
print(high)
```

OUTPUT

```
[500, 750, 1000]
```

Three more examples:

```
names = ["paracetamol", "aspirin"]
cleaned = [n.title() for n in names]
print(cleaned)
```

OUTPUT

```
['Paracetamol', 'Aspirin']
```

```
prices = [12.5, 8.0, 5.5]
with_tax = [round(p * 1.1, 2) for p in prices]
print(with_tax)
```

OUTPUT

```
[13.75, 8.8, 6.05]
```

```
temps = [36.5, 38.2, 37.0, 38.9]
fever_only = [t for t in temps if t > 37.5]
print(fever_only)
```

OUTPUT

```
[38.2, 38.9]
```

Read the last example aloud: *the temperatures that are above 37.5* — the comprehension is the sentence.

9.5 Nested lists — a table in a variable

A list can hold other lists — a natural way to represent a table, one inner list per row:

```
patients = [
    ["Sana", 32, 500],
    ["Ravi", 45, 250],
    ["Meera", 28, 750],
]
print(patients[0])          # the first row
```

```
print(patients[1][2])       # Ravi's dose
print(len(patients))        # number of patients
```

OUTPUT

```
['Sana', 32, 500]
250
3
```

Two indexes address one cell: `patients[1][2]` means row 1, column 2. This pattern scales straight into Unit IV's DataFrames — a DataFrame *is* a named, indexed table.

9.6 Tuples — the sealed record

A **tuple** is an ordered collection that, once created, **cannot be changed** — no appending, no replacing, no removing. It is written in round brackets (or just commas), and its immutability is a feature: fixed data — a blood-pressure reading, a date of birth, a set of coordinates — *should* resist accidental editing.

```
vitals = (120, 80)          # systolic, diastolic
systolic, diastolic = vitals    # unpacking
print(f"{systolic}/{diastolic} mmHg")
```

OUTPUT

```
120/80 mmHg
```

Feature	List	Tuple
Brackets	`[]`	`()`
Mutable?	yes — add, remove, change	no — sealed after creation
Speed	slower	faster
Best for	changing collections (stock)	fixed records (BP, DOB, coordinates)

Five examples:

```
vitals = (120, 80)
print(vitals[0], vitals[-1])
```

OUTPUT

```
120 80
```

```
single = (500,)           # note the comma - (500) is just an int!
print(type(single))
```

OUTPUT

```
<class 'tuple'>
```

```
bp_visits = [(120, 80), (128, 84), (135, 88)]
```

```
for sys_, dia in bp_visits:
    print(f"{sys_}/{dia} mmHg")
```

OUTPUT

```
120/80 mmHg
128/84 mmHg
135/88 mmHg
```

```
vitals = (120, 80, 120, 80)
print(vitals.count(120), vitals.index(80))
```

OUTPUT

```
2 1
```

```
# vitals[0] = 130     # TypeError - tuples are sealed
```

A list of tuples — `bp_visits` above — is the standard shape for repeated fixed readings: each visit is a sealed record, the collection of visits grows.

Chapter summary. Lists are numbered, changeable, mixed-type collections in `[]`; index from 0, slice with `[a:b]`, mutate with `append/insert/remove/pop/sort`. Copy with `.copy()`, never `=`. Comprehensions `[expr for item in list if cond]` build lists in one readable line. Nested lists form tables (`patients[1][2]`). Tuples are the immutable cousin in `()` — unpack them into named variables, use them for fixed records; a single item needs a trailing comma.

Quick checks 9. (a) `drugs = ["A", "B", "C"]; drugs[1:3]`? (b) Difference between `append` and `extend`? (c) What does `[d for d in [100, 200, 300] if d > 150]` produce? (d) Why does `(500)` not create a tuple? (e) Give one pharmacy situation for a list and one for a tuple.

Chapter 10 — Dictionaries and String Tools

10.1 Dictionaries — data with names on it

A **dictionary** holds information as **key : value** pairs within curly braces. Each key is unique and acts as the item's name — you look things up *by key*, not by position, exactly like finding a drug in an index by its name rather than by the page it happens to sit on.

```
drug_info = {"name": "Paracetamol", "dose_mg": 500, "category": "Analgesic"}
print(drug_info["name"])
print(drug_info["dose_mg"])
```

OUTPUT

```
Paracetamol
500
```

10.2 Adding, updating and removing

Dictionaries are mutable: assign to a key to create or update; pop to remove.

```
drug_info["stock"] = 120          # add a new key
drug_info["dose_mg"] = 650        # update an existing one
print(drug_info)
```

OUTPUT

```
{'name': 'Paracetamol', 'dose_mg': 650, 'category': 'Analgesic', 'stock': 120}
```

Five examples:

```
stock = {"Paracetamol": 120, "Ibuprofen": 45, "Aspirin": 12}
print(stock["Aspirin"])
```

OUTPUT

```
12
```

```
stock["Cetirizine"] = 80          # new entry
print(len(stock))
```

OUTPUT

```
4
```

```
stock.pop("Aspirin")              # remove
print("Aspirin" in stock)
```

OUTPUT

```
False
```

```
stock.update({"Ibuprofen": 60, "Paracetamol": 150})  # bulk update
print(stock["Ibuprofen"], stock["Paracetamol"])
```

OUTPUT

```
60 150
```

```
print(stock.get("Dolo", 0))       # safe lookup: 0 if key is absent
print(stock.get("Paracetamol"))
```

OUTPUT

```
0
150
```

`.get(key, default)` is the safe accessor — it returns the default (here 0) instead of raising `KeyError` when the key does not exist.

10.3 Looping through a dictionary

`.keys()`, `.values()` and `.items()` expose the three views; `.items()` paired with tuple unpacking is the everyday idiom:

```
stock = {"Paracetamol": 120, "Ibuprofen": 45, "Aspirin": 12}
for name, qty in stock.items():
    print(name, ":", qty, "units")
```

OUTPUT

```
Paracetamol : 120 units
Ibuprofen : 45 units
Aspirin : 12 units
```

10.4 Nested dictionaries — a small patient database

```
patients = {
    "P1": {"name": "Ravi", "age": 32, "dose": 500},
    "P2": {"name": "Sana", "age": 45, "dose": 250},
}
print(patients["P1"]["name"])
print(patients["P2"]["dose"])
```

OUTPUT

```
Ravi
250
```

One dictionary whose values are themselves dictionaries: each patient ID maps to a full record. This is the exact shape of a JSON record from a hospital API.

10.5 Dictionary comprehension

Same one-line idea as list comprehension, but building key:value pairs:

```
prices = {"Paracetamol": 12.5, "Ibuprofen": 8.0, "Aspirin": 5.5}
with_tax = {name: round(p * 1.1, 2) for name, p in prices.items()}
```

```
print(with_tax)
```

OUTPUT

```
{'Paracetamol': 13.75, 'Ibuprofen': 8.8, 'Aspirin': 6.05}
```

10.6 String manipulation techniques (syllabus point)

Unit I covered the core string methods. The table below collects the techniques this syllabus bullet expects you to command, each with a pharmacy-flavoured example:

Technique	Example	Result
f-string formatting	`f"Dose: {500} mg"`	`Dose: 500 mg`
`join()`	`", ".join(["Tab", "Cap"])`	`'Tab, Cap'`
`startswith()` / `endswith()`	`"Tab 123".startswith("Tab")`	`True`
`isdigit()`	`"500".isdigit()`	`True`
`title()`	`"paracetamol".title()`	`'Paracetamol'`
`splitlines()`	`"a\nb".splitlines()`	`['a', 'b']`
`.format()`	`"{} mg".format(500)`	`'500 mg'`
`zfill()`	`"7".zfill(3)`	`'007'`
`count()`	`"Paracetamol".count("a")`	`3`
`lstrip()` / `rstrip()`	`"  drug  ".lstrip()`	`'drug  '`

Worked example — processing a clinical note:

```
note = "Patient reports mild headache and nausea after dose"
words = note.split()
print("Word count:", len(words))
print("Contains nausea:", "nausea" in note)
cleaned = note.replace("mild", "moderate")
print(cleaned)
```

OUTPUT

```
Word count: 8
Contains nausea: True
Patient reports moderate headache and nausea after dose
```

Chapter summary. Dictionaries map unique keys to values in `{}`; access/update by key; `.get` for safe lookups; `.items()` for looping; nested dicts store records; comprehensions transform in one line. String tools — `join`, `startswith`, `isdigit`, `title`, `splitlines`, `zfill`, `format`, `count` — clean and shape pharmacy text; `split` turns a sentence into a word list; `replace` edits safely by returning a new string.

Quick checks 10. (a) `stock = {"A": 10, "B": 20}; stock["C"] = 30; len(stock)`? (b) Why is `.get()` safer than `[]`? (c) What does `", ".join(["a", "b"])` give? (d) `"123abc".isdigit()`? (e) Build a dict comprehension that doubles every value in `{"a": 1, "b": 2}`.

Chapter 11 — NumPy Arrays

11.1 What NumPy is and why it exists

NumPy (Numerical Python) is the third-party library that makes Python a serious scientific tool. Its core object is the **ndarray** — an n-dimensional array of numbers stored in contiguous memory and processed by compiled code. The practical consequence: operations that need a Python loop on a plain list happen in one vectorized expression on an array, tens to hundreds of times faster on large datasets. Every tool in Units IV–V (Pandas, Matplotlib) is built on NumPy arrays underneath.

Import with the universal alias:

```
import numpy as np
```

11.2 Creating arrays

Function	Creates	Example	Result
`np.array(list)`	from a list	`np.array([1, 2, 3])`	`array([1, 2, 3])`
`np.zeros(n)`	n zeros	`np.zeros(3)`	`array([0., 0., 0.])`
`np.ones(n)`	n ones	`np.ones(3)`	`array([1., 1., 1.])`
`np.arange(a, b)`	like range()	`np.arange(0, 10, 2)`	`array([0, 2, 4, 6, 8])`
`np.linspace(a, b, n)`	n evenly spaced	`np.linspace(0, 1, 5)`	`array([0., 0.25, 0.5, 0.75, 1.])`

Five examples:

```
doses = np.array([500, 750, 1000])
print(doses)
print(doses.shape)
print(doses.dtype)
```

OUTPUT

```
[ 500  750 1000]
(3,)
int64
```

`.shape` gives the dimensions as a tuple (here, 3 elements in one dimension); `.dtype` reports the element type — `int64` means 64-bit integers.

```
print(np.zeros(5))
```

OUTPUT

```
[0. 0. 0. 0. 0.]
```

```
print(np.arange(0, 10, 3))
```

OUTPUT

```
[0 3 6 9]
```

```
weights = np.array([60, 70, 55])
doses = weights * 5              # element-wise multiply
print(doses)
```

OUTPUT

```
[300 350 275]
```

```
doses_2d = np.array([[250, 500], [300, 600], [200, 400]])
print(doses_2d.shape)
print(doses_2d[1, 0])            # row 1, column 0
```

OUTPUT

```
(3, 2)
300
```

The 2-D example is a 3-row × 2-column matrix — the same `patients` table idea from §9.5, now with fast maths.

11.3 Element-wise arithmetic — the vectorized advantage

On a plain list, adding 5 to every dose requires a loop. On a NumPy array, `arr + 5` does it in one expression — and every arithmetic operator works this way (element-wise):

```
doses = np.array([500, 750, 1000])
print(doses + 100)
print(doses * 2)
print(doses / 2)
```

OUTPUT

```
[ 600  850 1100]
[1000 1500 2000]
[250. 375. 500.]
```

Note division yields floats — the same `/`-always-floats rule from Unit I, now applied to whole arrays at once.

11.4 Aggregation functions

Function	Purpose
`.mean()`	average of all elements
`.sum()`	total
`.max()` / `.min()`	largest / smallest
`.std()`	standard deviation

```
doses = np.array([500, 750, 1000, 1250])
print(doses.mean())
print(doses.sum())
print(doses.max(), doses.min())
```

OUTPUT

```
875.0
3500
1250 500
```

11.5 Slicing and boolean filtering

Slicing works exactly like list slicing:

```
doses = np.array([500, 750, 1000, 1250])
print(doses[1:4])
```

OUTPUT

```
[ 750 1000 1250]
```

Boolean filtering is the NumPy superpower: a comparison on an array yields an array of True/False; using that as an index selects only the True elements:

```
print(doses > 600)
print(doses[doses > 600])
```

OUTPUT

```
[False  True  True  True]
[1000 1250]
```

More examples:

```
temps = np.array([36.5, 38.2, 37.0, 38.9, 36.8])
print(temps[temps > 37.5])
```

OUTPUT

```
[38.2 38.9]
```

```
bp = np.array([118, 125, 160, 110, 145])
print(np.sort(bp))
print(np.where(bp > 130))
```

OUTPUT

```
[110 118 125 145 160]
(array([2, 4]),)
```

`np.where(condition)` returns the *positions* where the condition holds — here, indices 2 and 4 carry readings above 130. Together with boolean filtering (`bp[bp > 130]` gives the *values*), these two cover most screening questions a pharmacist will ask of an array.

```
weights = np.array([60, 70, 55, 80])
safe = weights[(weights >= 50) & (weights <= 75)]
print(safe)
```

OUTPUT

```
[60 70 55]
```

(Note: NumPy uses `&` and `|` for element-wise and/or, with brackets around each comparison.)

Chapter summary. `import numpy as np`; create arrays with `np.array` / `zeros` / `ones` / `arange` / `linspace`; check `.shape` and `.dtype`. Arithmetic is element-wise (`arr + 5`, `arr * 2`); aggregation with `.mean()`, `.sum()`, `.max()`, `.min()`. Slicing copies list behaviour; boolean filtering `arr[arr > limit]` selects matching elements and `np.where` finds their positions — the everyday tools for screening health data at speed.

Quick checks 11. (a) `np.arange(0, 6, 2)`? (b) What does `doses.shape` return for a 1-D array of 4 elements? (c) `np.array([10, 20, 30]).mean()`? (d) What does `temps[temps > 37]` do? (e) Why is `arr * 2` faster than a loop?

Chapter 12 — CSV Files and Healthcare Datasets

12.1 What CSV is

CSV (Comma-Separated Values) is the plain-text interchange format for tables: every line holds one row, and commas split it into columns. Hospital systems, lab instruments and spreadsheet software all export CSV — it is the lingua franca of data exchange.

A small patient file, `patients.csv`, might look like this in plain text:

```
Name,Age,Dose_mg
Ravi,32,500
Sana,45,250
Meera,28,750
```

12.2 Reading a CSV file — the csv module

Python's built-in `csv` module parses the commas for you. The pattern: `open` the file, wrap it in `csv.reader`, loop the rows (each row arrives as a **list of strings**):

```
import csv
with open("patients.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

OUTPUT

```
['Name', 'Age', 'Dose_mg']
['Ravi', '32', '500']
['Sana', '45', '250']
['Meera', '28', '750']
```

Two pitfalls the syllabus expects you to know:

1. **Everything arrives as text** — `'500'` is a string. Cast with `int()` or `float()` before any arithmetic.
2. **On Windows, always pass `newline=""` when writing** — forgetting it inserts blank lines between rows.

12.3 Writing a CSV file

```
import csv
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age", "Dose_mg"])
    writer.writerow(["Ravi", 32, 500])
    writer.writerow(["Sana", 45, 250])
```

The file `output.csv` now contains the same three lines shown in §12.1.

12.4 DictReader — rows as dictionaries

`csv.DictReader` reads the header line as column names, then delivers every following row as a **dictionary** keyed by those names — usually more convenient than positional lists:

```
import csv
with open("patients.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["Name"], row["Dose_mg"])
```

OUTPUT

```
Ravi 500
Sana 250
Meera 750
```

12.5 Handling a missing file safely

```
try:
    with open("patients.csv", "r") as file:
        data = file.read()
    print("File loaded:", len(data), "characters")
except FileNotFoundError:
    print("Error: patients.csv not found")
```

OUTPUT

```
Error: patients.csv not found
```

(`try/except` catches the error and prints a friendly message instead of crashing — a preview of defensive programming.)

12.6 Structured healthcare datasets (syllabus point)

A **structured dataset** arranges information into **rows** (each row one patient, sample or observation) alongside **columns** (each column one attribute) — the same arrangement as a spreadsheet or database table:

Column	Example	Data type
Patient_ID	P1024	string
Age	45	integer
Drug_Name	Metformin	string
Dose_mg	500	integer
BP_Systolic	128	integer
Adverse_Reaction	False	boolean

The dataset's **shape** is (rows, columns) — `(500, 6)` means 500 patient records with 6 fields each. Structure matters because it enables filtering, sorting and grouping ("find all patients above 60 on Metformin") — operations that are nearly impossible on unorganised text.

Dirty data preview (fully treated in Unit IV): missing values (`NaN`), wrong types (`'500'` stored as text), duplicate records — with Pandas fixes `dropna()`, `astype(int)`, `drop_duplicates()`.

12.7 Importing a pharmaceutical dataset — the complete workflow

Combining everything in this unit, here is a small but complete analysis on a pharmacy stock dataset stored as CSV:

```
import csv

# Build a list of dictionaries from the file
inventory = []
with open("drug_stock.csv", "r") as file:
    for row in csv.DictReader(file):
        inventory.append({
            "name": row["Drug_Name"],
            "qty": int(row["Stock_Qty"]),
            "price": float(row["Price"]),
        })

# Low-stock report
low_stock = [item["name"] for item in inventory if item["qty"] < 20]
print("Low stock:", low_stock)

# Total inventory value
total_value = sum(item["qty"] * item["price"] for item in inventory)
print(f"Total value: Rs. {total_value:.2f}")
```

OUTPUT

```
Low stock: ['Aspirin', 'Cetirizine']
Total value: Rs. 344.40
```

Every line of that program is a Unit III skill: `open` + `csv.DictReader` reads the file; the comprehension filters; `sum` with a generator expression totals; the f-string formats. Change the file, re-run, and the report updates — the shape of every data pipeline in Unit IV and beyond.

Chapter summary. CSV is plain-text tabular data — the exchange format of healthcare systems. `csv.reader` yields rows as string lists (cast numbers before maths); `csv.writer` with `newline=""` writes; `DictReader/DictWriter` key rows by header names; `try/except FileNotFoundError` guards missing files. A structured dataset has rows (records) and columns (fields) with a (rows, columns) shape; missing values, wrong types and duplicates are the three standard dirt problems. The §12.7 workflow — read → cast → filter → total — is the template for every small pharmacy dataset task.

Quick checks 12. (a) Why must CSV numbers be cast before arithmetic? (b) What does `newline=""` prevent? (c) `DictReader` vs `reader` — one advantage? (d) A dataset with 200 rows and 8 columns has what shape? (e) Name three data-quality problems and their Pandas fixes.

Unit III — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Lists, tuples, and dictionaries	Ch 9–10
Indexing and slicing lists; basic operations on lists and dictionaries	Ch 9 (§9.2–9.3), Ch 10 (§10.2–10.3)
String manipulation techniques	Ch 10 (§10.6)
Introduction to NumPy arrays; basic operations	Ch 11
Reading and writing CSV files	Ch 12 (§12.1–12.5)
Understanding structured healthcare datasets; importing pharmaceutical datasets	Ch 12 (§12.6–12.7)

Twenty-question exercise bank

Predict the output (1–8).

1. `drugs = ["A", "B", "C"]; drugs.append("D"); print(len(drugs))`
2. `print([10, 20, 30][::-1])`
3. `vitals = (120, 80); print(vitals[-1])`
4. `d = {"x": 1, "y": 2}; d["z"] = 3; print(len(d))`
5. `import numpy as np; print(np.array([1, 2, 3]) * 3)`
6. `print(np.arange(0, 10, 5))`
7. `temps = np.array([36, 38, 37, 39]); print(temps[temps > 37])`
8. `print("  strip me  ".strip())`

Short answers (9–14).

9. List vs tuple — two differences and one use-case each.
10. Why does `new = old` not copy a list? Show the correct way.
11. What is a list comprehension? Give one with a condition.
12. Boolean filtering in one sentence, with a temperature example.
13. Why is a dictionary better than parallel lists for a patient record?
14. Name three data-quality problems in healthcare CSV files.

Write programs (15–20).

15. Build a list of 5 drug names; print the first, last, and sorted list.
16. Count fevers in a list of temperatures using a comprehension.

17. Create a dictionary for one drug (name, dose, stock); add a price key; loop `.items()`.
18. `np.linspace(0, 10, 5)` — what array? Then compute its mean.
19. Read `patients.csv` with DictReader and print each patient's name and dose.
20. Given a list of tuples `[(120, 80), (130, 85)]`, unpack and print each as `sys/dia mmHg`.

Answers to selected exercises

1. `4`. 2. `[30, 20, 10]`. 3. `80`. 4. `3`. 5. `[3 6 9]`. 6. `[0 5]`. 7. `[38 39]`. 8. `strip me`. 10. The two names share one list object; use `old.copy()`. 11. `[expr for x in list if cond]`; e.g. `[t for t in temps if t > 37.5]`. 12. `arr[arr > limit]` selects only elements above the limit. 13. Keys are self-describing (`p["dose_mg"]` vs `row[2]`); no risk of column-order drift. 18. `[0. 2.5 5. 7.5 10.]`, mean `5.0`.

Important points for examination

1. A **list** (`[]`) is ordered and mutable; a **tuple** (`()`) is ordered and immutable; a **dict** (`{}`) maps unique keys to values and is mutable.
2. List indexing starts at 0; slices `[a:b]` exclude b; `[::-1]` reverses; `IndexError` on reading past `len-1`.
3. Core list methods: `append` (end), `insert` (position), `remove` (value), `pop` (position, returns item), `sort`, `reverse`, `count`, `index`, `extend`, `clear`.
4. `new = old` aliases a list; `.copy()` creates an independent one — a classic examination trap.
5. A list comprehension `[expr for item in collection if condition]` builds a list in one readable line; the same idea works for dictionaries.
6. A tuple of one item needs a trailing comma: `(500,)`.
7. Dict access by key `d["k"]`; safe access with `d.get("k", default)`; loop with `for k, v in d.items()`.
8. Useful string tools: `join`, `startswith/endswith`, `isdigit`, `title`, `splitlines`, `zfill`, `format`, `count`, `lstrip/rstrip` — beyond the Unit I core of `split`, `strip`, `replace`, `find`, `in`.
9. NumPy arrays support **element-wise** arithmetic (`arr * 2`, `arr + 5`) — vectorization replaces loops and is dramatically faster.
10. Create arrays with `np.array`, `np.zeros`, `np.ones`, `np.arange`, `np.linspace`; inspect with `.shape` and `.dtype`; aggregate with `.mean()`, `.sum()`, `.max()`, `.min()`.
11. Boolean filtering `arr[arr > limit]` selects matching elements; `np.where(condition)` gives their positions.
12. CSV = comma-separated plain text. `csv.reader` yields string-list rows (cast numbers!); `csv.writer` needs `newline=""` on Windows; `DictReader` keys rows by header.

13. A structured dataset is rows × columns; its shape is `(rows, columns)`; typical dirt: missing values, wrong types, duplicates.
14. The standard workflow: read with DictReader → cast to int/float → filter with a comprehension → aggregate with sum or a NumPy method — exactly the §12.7 inventory report.

Sources for this unit: course decks and the book plan's source matrix; topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 3* notes, MCA R24 lecture notes Unit II (data structures section), Yamuna Devi, Jeyabharathi & Vajpayee *Python Programming* (GCS), MRCET R17 notes, numpy.org tutorials, and the curated learn.biodockify.com catalog (numpy, pandas, trekhleb/learn-python, exercism/python, W3Schools). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source (zero 7-word overlaps). Full credits: see the book's Sources and Acknowledgements page.